

1. Simulation

Since the meshing component supports export to multiple standard formats, the Workbench can interface with a wide range of commercial and open-source simulation packages. This flexibility allows users to integrate different numerical solvers into the modeling workflow depending on the problem requirements and available software infrastructure.

In the current version of the Workbench, SfePy (Simple Finite Elements in Python), an open-source, Python-based multi-physics finite element framework, is integrated.

The solver can directly import meshes generated within the Workbench. In the following sections, we describe how to set up and run a simulation using this software package. To demonstrate the workflow and illustrate the integration process, we consider a simple hydrothermal benchmark model as an example. This example highlights the complete procedure—from importing generated mesh and parameter definition to solver execution and result visualization—within the automated Workbench environment.

1.1. Running simulations

1.1.1. *SfePy* (*simulation_run()*)

SfePy uses a *Python* input file with the extension *.py* to define the complete simulation setup, including the mesh, regions, materials, fields, variables, equations, solvers, and output settings. A detailed description of the input file structure and available options can be found in the official *SfePy* user guide and manual available on the *SfePy* website.

To run an *SfePy* model within the Workbench, certain parts of the input file must be defined in a way that allows dynamic interaction with the Workbench environment.

First, the mesh definition must read the mesh filename from an environment variable. This enables the Workbench to pass the automatically generated mesh to *SfePy* at runtime. The following lines should be included in the input file:

```
import os
import meshio
filename_mesh = os.environ.get("TEMP_MESH_FILE", "filename.mesh")
```

Here, "filename.mesh" acts as a fallback default value. During execution through *simulation_run()*, the Workbench sets the environment variable *TEMP_MESH_FILE* to the actual mesh file exported from the meshing module. This ensures that the solver always uses the correct, automatically generated mesh without manual modification of the input script.

Second, the output directory must also be defined dynamically to ensure that results are written to the appropriate project folder created by the Workbench. This can be achieved as follows:

```
output_dir = os.environ.get("SFEpy_OUTPUT_DIR", "output")
os.makedirs(output_dir, exist_ok=True)
```

Again, "output" serves as a default directory if the environment variable is not set. When the model is executed from the Workbench, the *SFEpy_OUTPUT_DIR* variable is automatically defined, ensuring that all result files are stored in the designated simulation folder.

In addition, the output directory must be referenced in the solver options section of the *SfePy* input file. For example:

```

options = {'nls': 'newton',
'ls': 'ls',
'ts': 'ts',
'save_times': 'all',
'output_dir': output_dir,
}

```

By linking *output_dir* in the options dictionary, *SfePy* writes all result files into the *output_dir* managed by the Workbench.

With these two dynamic components—the mesh filename and the output directory—the *SfePy* input file becomes fully compatible with the automated Workbench workflow. The remaining parts of the input file, such as material properties, boundary conditions, governing equations, and solver settings, can be defined in the standard *SfePy* format according to the specific hydrothermal or multi-physics problem under consideration.

The function *run_sfepy()* is designed to execute an *SfePy* simulation within the Workbench environment using an automatically generated mesh. It handles mesh conversion, environment configuration, solver execution, output management, and cleanup in a structured and automated way.

The function takes three arguments: *input_file*, which is the *SfePy Python* input script; *mesh_test*, which represents the internal mesh object generated by the Workbench; and an optional *output_dir*, which defines where simulation results should be stored.

The first part of the function manages the output directory. If no output directory is provided, a temporary directory is created using *Python*'s *tempfile* module. In this case, a flag (*auto_output*) is set to indicate that the directory was automatically generated. If the user provides an output directory, the function ensures that it exists by creating it if necessary. The selected output directory is then passed to *SfePy* via the environment variable *SFEpy_OUTPUT_DIR*, allowing the input file to dynamically retrieve and use this location for writing results.

Next, the mesh generated inside the Workbench is exported to *Exodus* format using the *export_mesh_results_to_exodus()* function. The exported mesh data is written into a temporary *.exo* file on disk. After exporting the mesh, the function converts the *Exodus* file into a *SfePy*-compatible format using the *meshio* library. The mesh is read from the temporary *Exodus* file, and material IDs are assigned to each cell block. These material identifiers are stored in the *cell_data* dictionary under the key "mat id", which allows *SfePy* to distinguish between different material regions during simulation. The mesh is then written to a second temporary file in the *Medit .mesh* format, which *SfePy* can directly read. The path to this temporary mesh file is passed to the solver via the environment variable *TEMP_MESH_FILE*.

Once the mesh preparation is complete, the function launches the *SfePy* simulation using the subprocess module. The command "*Sfepy-run input_file*" is executed in the current working directory, and the process waits until the simulation finishes. Because the required environment variables were set earlier, the input file can dynamically retrieve both the mesh file and the output directory.

After the simulation completes, the function lists all files generated in the output directory. This provides immediate feedback about the produced result files, such as *VTK* outputs.

In the final stage, the function performs cleanup operations. Temporary helper files, including the temporary *Exodus* file and the temporary *Medit* mesh file, are removed to avoid leaving unnecessary intermediate files on the system. The output directory itself is only removed if it was automatically created; if the user explicitly provided an output directory, it is preserved.

Finally, the function returns the path to the output directory, allowing further post-processing or visualization steps within the Workbench workflow.

1.2. Storing outputs in a uniform format

Different simulation packages export results in different file formats and internal data structures. For consistent post-processing, visualization, and comparison of results within the Workbench, it is therefore essential to unify these heterogeneous outputs into a common internal representation.

Currently, the Workbench supports two major output formats: *Exodus* and *VTK*. Since these formats differ in structure, metadata handling, and storage of time-dependent fields, their contents are converted into a standardized data container. This ensures that all downstream processes, i.e., visualization and analysis, operate independently of the original solver and file format.

To achieve this, simulation outputs are stored in a unified data class called *SimulationResults*. This class acts as a time-dependent container for geometry, topology, and associated field variables.

The *SimulationResults* class stores data per timestep using dictionaries keyed by time. Each timestep contains the complete mesh description and all associated result fields. The structure is defined as follows:

- *nodes_by_time*: stores the nodal coordinates for each timestep as a *NumPy* array of shape $(n_nodes, 3)$.
- *cells_by_time*: stores the cell connectivity for each timestep.
- *celltypes_by_time*: stores the *VTK*-style cell type identifiers corresponding to each element.
- *node_data_by_time*: stores node-based variables (e.g., temperature, pressure, displacement) as dictionaries mapping variable names to *NumPy* arrays for each timestep.
- *cell_data_by_time*: stores element-based variables (e.g., stress, material IDs, fluxes) in a similar dictionary structure.

By organizing the data in this way, the Workbench provides a solver-independent internal representation of simulation results. Whether the original output file is in *Exodus* or *VTK* format, it is parsed and mapped into this standardized structure. This approach offers several advantages. It decouples post-processing routines from solver-specific formats, simplifies the implementation of visualization modules, enables consistent time-series handling, and facilitates comparison between different simulation engines.

In the following sections, the procedures for reading and converting *Exodus* and *VTK* outputs into the *SimulationResults* structure are described in detail.

1.2.1 Importing mesh from *VTK* files (*load_vtk_results()*)

The function *load_vtk_results()* is responsible for reading simulation results stored in *VTK* format from a given output directory and converting them into the unified *SimulationResults* data structure used by the Workbench.

At the beginning, the provided output directory path is converted into an absolute path to ensure consistency and avoid potential issues related to relative paths. The function then recursively scans the directory and all its subdirectories to locate files with the *.vtk* extension. All detected *VTK* files are collected into a list, which is subsequently sorted. Sorting ensures that the files are processed in a consistent and predictable order, typically corresponding to increasing simulation time.

A new *SimulationResults* object is then created to store the extracted data. For each *VTK* file found in the directory, the function attempts to extract the simulation time from the filename. It does this by splitting the filename and interpreting the second-to-last segment as a floating-point number. If this conversion fails, the filename itself is used as the time identifier. This mechanism allows the function to handle both time-stepped outputs (for example, *solution.0.0.vtk*, *solution.1.0.vtk*) and non-standard naming conventions.

Each *VTK* file is read using *PyVista*. From the loaded mesh object, the function extracts the nodal coordinates, cell connectivity, and cell type information. These geometric and topological components are stored in the corresponding dictionaries of the *SimulationResults* instance, using the extracted time value as the key. In addition to the mesh structure, all point-based data arrays are collected from the *VTK* file and stored in the *node_data_by_time* dictionary. Similarly, all cell-based data arrays are stored in the *cell_data_by_time* dictionary. Each dataset is converted into a *NumPy* array to ensure consistent numerical handling within the Workbench.

After all *VTK* files have been processed, a confirmation message is printed indicating successful loading.

Finally, the function includes an automatic cleanup mechanism. It checks whether the output directory is located inside the system's temporary directory. If this is the case, the directory is removed after loading the data. This behavior prevents the accumulation of temporary simulation outputs created during automated runs. If the directory is not part of the system's temporary path, it is preserved, assuming it was intentionally specified by the user.

The function then returns the populated *SimulationResults* object, which provides a standardized, time-dependent representation of the *VTK* simulation results for further post-processing and visualization within the Workbench.

1.2.2 Importing mesh from *Exodus* files (*load_exodus_results()*)

The function *load_exodus_results()* is responsible for reading simulation results stored in *Exodus II* format from a given file and converting them into the unified *SimulationResults* data structure used by the Workbench.

At the beginning, the provided file path is converted into an absolute path to ensure consistency and avoid potential issues related to relative paths. A message is printed to indicate which *Exodus* file is being loaded.

The function then creates a *vtkExodusIIReader* object, which is responsible for reading *Exodus* files. *UpdateInformation()* is called to read metadata from the file, such as the available point (nodal) and element (cell) variables and the timesteps, without loading all the data yet.

Next, all point and cell result arrays are enabled using loops over the reader's available arrays. This ensures that every variable stored in the *Exodus* file is read. The reader is also configured to generate unique global IDs for nodes and elements, as well as an object ID array for cells, which helps in tracking entities consistently across timesteps.

The function retrieves all available timesteps from the *Exodus* file and prints them. A new *SimulationResults* object is created to store the extracted data.

The function then loops over each timestep. For each time step, the reader is updated to that specific time using *VTK*'s streaming pipeline mechanism. The output of the reader is wrapped as a *PyVista* object for easier manipulation.

Since *Exodus* files can contain nested multi-block structures, a helper function, *iter_leaf_blocks* is used to recursively extract all leaf meshes, which are the actual mesh pieces containing points and cells. Any empty meshes are ignored. If no valid meshes are found for a timestep, an error

is raised. The collected leaf meshes are then combined into a single mesh for that timestep using *PyVista*'s *combine()* method.

From the combined mesh, the function extracts nodal coordinates, cell connectivity, and cell type information. These geometric and topological components are stored in the corresponding dictionaries of the *SimulationResults* instance using the timestep as the key. Additionally, all point-based (nodal) and cell-based (element) variable arrays are collected and converted into *NumPy* arrays for consistent numerical handling. These are stored in the *node_data_by_time* and *cell_data_by_time* dictionaries of the *SimulationResults* object.

After processing all timesteps, a confirmation message is printed indicating that all data has been successfully loaded.

Finally, the function includes an optional cleanup mechanism. It checks whether the *Exodus* file resides in the system's temporary directory. If this is the case, the entire directory containing the file is removed after reading, which prevents the accumulation of temporary simulation outputs. If the file is not in a temporary directory, it is preserved, assuming it was intentionally specified by the user.

The function then returns the fully populated *SimulationResults* object, which provides a standardized, time-dependent representation of the *Exodus* simulation results for further post-processing, analysis, or visualization within the Workbench.

1.3. Quick visualization/inspection

Besides visualization within *LiquidEarth*, numerical results can also be explored using several quick visualization/inspection utilities. These tools allow users to:

- Plot a field variable at a given time step
- Plot a variable on a cross-section at a given time
- Plot a variable along a specified line at a given time
- Plot the time series of a variable at a given spatial location
- Print the magnitude of a variable at a specific location and time

To generate plots of a field variable at a given time — whether on the full domain, on a cross-section, or along a line — a *PyVista* mesh object must first be reconstructed from the stored simulation results.

The function *build_grid_from_class()* performs this reconstruction. It converts the internal *SimulationResults* data structure into a *PyVista* mesh object corresponding to a selected simulation time.

The function takes two input arguments: *sim*, a *SimulationResults* object containing the stored geometry and variable data for all timesteps of a simulation; and *time*, a floating-point value representing the specific simulation time step to be reconstructed.

The function first verifies that the requested time step exists in the stored results. If the specified time is not available in *sim.nodes_by_time*, a *ValueError* is raised to prevent reconstruction of a non-existent state.

It then retrieves the nodal coordinates, element connectivity, and cell type information associated with that time step from the *SimulationResults* object. The nodal coordinates define the geometry of the mesh, while the cell connectivity and cell types define its topology.

If no cell connectivity information is present (i.e., if *cells* is *None* or empty), a *PolyData* object is created using only the nodal coordinates. This represents a point-based dataset without full element topology. Otherwise, a complete *UnstructuredGrid* object is constructed using the cells, cell types, and node coordinates, representing the full finite element mesh.

Once the geometric structure has been reconstructed, all node-based variables stored in *sim.node_data_by_time* for the selected time step are attached to the mesh as point data arrays.

Similarly, all cell-based variables stored in *sim.cell_data_by_time* are attached as cell data arrays.

The function then returns the fully reconstructed *PyVista* mesh corresponding to the requested time step. This mesh contains both the geometry and all associated simulation variables, making it ready for visualization, analysis, and further post-processing.

1.3.1 Plotting a variable at a given time (*plot_variable_at_a_time()*)

The function *plot_variable_at_a_time()* provides a quick 3D visualization of a selected simulation variable at a specific time step using *PyVista*.

The input parameters are: *sim* which is a *SimulationResults* object that contains the mesh and associated result data for all time steps; *var_name*, a string specifying the name of the variable to be visualized (for example displacement, stress, temperature, etc.); *time*, the specific time step at which the variable should be plotted; *cmap*, which is an optional argument specifying the colormap used for visualization (default is "viridis"); *show_edges*, a boolean that controls whether mesh edges are displayed in the plot, which can help in visualizing element boundaries; and *scale*, a tuple of three scaling factors applied to the *x*, *y*, and *z* coordinates respectively, allowing geometric scaling of the model before visualization.

The function first builds the mesh corresponding to the requested time step by calling *build_grid_from_class()*. This reconstructs the *PyVista* grid including geometry and associated data arrays for that specific time. It then checks whether the requested variable exists in either the point data (*grid.point_data*) or the cell data (*grid.cell_data*). If the variable is not found, a *ValueError* is raised to prevent plotting invalid data.

For visualization, the geometric surface of the mesh is extracted using *extract_geometry()*. This ensures that only the outer surface of the model is rendered, which improves clarity and performance for volumetric meshes. The surface coordinates are then scaled by multiplying them with the provided scale factors, allowing optional geometric exaggeration or unit adjustments.

A *PyVista* Plotter object is created to manage the rendering window. The surface is added to the plot using the selected scalar variable and colormap. If *show_edges* is enabled, element edges are displayed on top of the surface. A scalar bar is added to indicate the range and distribution of the selected variable. Additionally, a descriptive title showing the variable name and time step is displayed at the top of the visualization window.

Finally, axes and a background grid are shown to provide spatial reference, and the plot is rendered.

Note: Example of using *plot_variable_at_a_time()*:

```
plot_variable_at_a_time(data_by_time, "T", 0, cmap="coolwarm", show_edges=True  
scale=(1,1,1))
```

Here, *data_by_time* is a *SimulationResults* object in which the simulation results are stored (for example, *data_by_time = load_vtk_results(output_dir)*). The function plots the temperature field (indicated by "T") at time step "0". The "coolwarm" colormap is used for visualization, and mesh edges are displayed in the plot. A uniform scaling factor of 1 is applied in all three spatial dimensions (*x*, *y*, *z*).

1.3.2 Plotting a variable on a cross-section at a given time (*plot_cross_section()*)

The function *plot_cross_section()* visualizes a cross-sectional slice of a selected simulation variable at a specific time step using *PyVista*. It allows inspection of the internal distribution of a variable by cutting the 3D model with a plane.

The input parameters are: *sim*, a *SimulationResults* object containing the mesh and associated result data for all time steps; *var_name*, a string defining the name of the variable to visualize; *time*, specifying the time step at which the variable should be evaluated; *origin*, defining the slicing plane: origin is a 3D point through which the plane passes; *normal*, a 3D vector defining the plane's orientation; *cmap*, an optional parameter which defines the colormap used for visualization (default is "viridis"); *show_edges*, a boolean controlling whether element edges are displayed on the slice; and *scale*, a tuple of scaling factors applied to the *x*, *y*, and *z* coordinates of the sliced geometry.

The function first reconstructs the mesh for the selected time step using *build_grid_from_class()*. It then verifies that the requested variable exists either in the point data (*grid.point_data*) or the cell data (*grid.cell_data*). If the variable is not found, a *ValueError* is raised to prevent invalid visualization.

A planar slice of the mesh is created using *grid.slice()*. This operation cuts the mesh with the defined plane and returns the resulting intersection geometry. The coordinates of the sliced mesh are optionally scaled using the provided scaling factors.

A *PyVista* *Plotter* object is then created for rendering. If the slicing operation results in a dataset with no cells (for example, if the slice only intersects points), the function adjusts rendering settings to display points as spheres with a defined size. This ensures that even degenerate or very thin intersections remain visible.

The sliced mesh is added to the plot using the selected scalar variable and colormap. A horizontal scalar bar is added to show the value range of the variable. A descriptive title indicating the variable name and time step is displayed at the top of the window. Axes and a background grid are shown to provide spatial reference. Finally, the plot is rendered interactively.

Note: Example of using *plot_cross_section()*:

```
plot_cross_section(data_by_time,"p", 0, origin=(540,20,100), normal=(1,0,0),  
cmap="coolwarm", show_edges=True, scale=(1,1,1))
```

Here, *data_by_time* is a *SimulationResults* object in which the simulation results are stored (for example, *data_by_time = load_vtk_results(output_dir)*). This command plots a cross-sectional slice of the variable "p" from the simulation results stored in *data_by_time* at time step "0". The slice is made through the point (540, 20, 100) with a plane oriented perpendicular to the x-axis (1, 0, 0), showing the internal distribution of "p" within the 3D model. The "coolwarm" colormap is used for visualization, and mesh edges are displayed in the plot. A uniform scaling factor of 1 is applied in all three spatial dimensions (*x*, *y*, *z*).

1.3.3 Plotting a variable along a line at a given time (*plot_variable_along_line()*)

The function *plot_variable_along_line()* visualizes the variation of a selected simulation variable along a straight line between two points at a specific time step. Instead of showing a 3D field, it produces a 2D plot of the variable value versus distance along the line, which is useful for profile analysis and quantitative evaluation.

The input parameters are: *sim*, a *SimulationResults* object containing the mesh and associated result data for all time steps; *var_name*, a string specifying the name of the variable to be evaluated; *time*, defining the time step at which the variable should be sampled; *p0* and *p1*, the 3D coordinates of the start and end points of the sampling line; and *n_samples*, defining the number of subdivisions along the line (default is 200), controlling the resolution of the sampling.

The function first reconstructs the mesh for the requested time step using *build_grid_from_class()*. It then checks whether the specified variable exists in either the point data or cell data of the grid. If the variable is not found, a *ValueError* is raised to prevent invalid sampling.

A straight line between *p0* and *p1* is created using *PyVista* with the specified resolution. The line is then sampled against the simulation grid using *line.sample()*. This interpolates the variable values from the mesh onto the points along the line. If the sampling operation produces no points (for example, if the line lies completely outside the mesh domain), a *RuntimeError* is raised to indicate that the chosen line does not intersect the model.

From the sampled result, the variable values are extracted together with the automatically computed *Distance* array, which represents the cumulative distance from the starting point along the line. These two arrays are then plotted using *Matplotlib*, with distance on the horizontal axis and the variable values on the vertical axis. The plot includes axis labels, a descriptive title indicating the variable and time step, and a grid for readability.

Note: Example of using *plot_variable_along_line()*:

```
plot_variable_along_line(data_by_time,"p", 0, p0=(500,20,0), p1=(500,20,1000),
n_samples=20)
```

Here, *data_by_time* is the data class in which the simulation results are stored (for example, *data_by_time = load_vtk_results(output_dir)*). This command plots the variation of the variable "p" at time step "0" along a line specified by *p0* and *p1* with the number of sampling of 20.

1.3.4 Printing a variable at a Point at a given time (*print_variable_at_point()*)

The function *print_variable_at_point()* evaluates and prints the value of a selected simulation variable at a specific spatial point and time step. It is designed for quick numerical inspection rather than graphical visualization.

The input parameters are: *sim*, a *SimulationResults* object containing the mesh and associated result data for all time steps; *var_name*, a string specifying the name of the variable to evaluate; *time*, defining the time step at which the variable should be accessed; and *point*, a 3D coordinate (x, y, z) representing the spatial location where the variable should be sampled.

The function first reconstructs the mesh corresponding to the selected time step by calling `build_grid_from_class()`. It then verifies that the requested variable exists either in the point data (`grid.point_data`) or cell data (`grid.cell_data`). If the variable is not available, a `ValueError` is raised to prevent invalid evaluation.

To evaluate the variable at the specified location, the function creates a `PyVista PolyData` object containing the single query point. This point dataset is then sampled against the simulation grid using `sample()`. This operation interpolates the variable value from the surrounding mesh elements to the given point location.

If the sampling result contains no points, it indicates that the query point lies outside the mesh domain. In that case, a warning message is printed showing the point coordinates and the mesh bounds, and the function returns `None`.

If the point lies inside the mesh, the interpolated value of the requested variable is extracted and printed together with the point coordinates and time step.

Note: Example of using `print_variable_at_point()`:

```
print_variable_at_point(data_by_time, "p", 0, point=(500,20,500))
```

Here, `data_by_time` is the data class in which the simulation results are stored (for example, `data_by_time = load_vtk_results(output_dir)`). This command prints the value of the variable "p" from the simulation results stored in `data_by_time` at time step "0" at the location of (500, 20, 500).

1.3.5 Plotting time Series at a Point (`plot_variable_time_series()`)

The function `plot_variable_time_series()` visualizes the temporal evolution of a selected simulation variable at a fixed spatial point. Instead of showing spatial distribution at one time step, this function tracks how the variable changes over all available time steps and presents the result as a 2D time-history plot.

The input parameters are: `sim`, a `SimulationResults` object that stores mesh and result data organized by time steps.; `var_name`, a string specifying the name of the variable to evaluate; `point`, a 3D coordinate (x, y, z) representing the spatial location where the variable should be monitored over time; `decimals`, an optional parameter controlling the number of decimal places displayed on the y-axis of the plot (default is 3).

The function first retrieves and sorts all available time steps from `sim.nodes_by_time.keys()`. It then initializes an empty list to store the sampled values. For each time step, the mesh is reconstructed using `build_grid_from_class()`. The function checks whether the requested variable exists in either the point data or the cell data of the grid. If the variable is missing at any time step, a `ValueError` is raised.

To evaluate the variable at the specified point, a `PyVista PolyData` object containing the single query point is created and sampled against the grid. This interpolates the variable value from the surrounding mesh elements. If the point lies outside the mesh at a given time step, no sampled points are returned, and a `NaN` value is appended to the list to indicate missing data. Otherwise, the interpolated value is stored.

After looping over all time steps, the collected values are converted into a `NumPy` array. A `Matplotlib` figure is then created to plot the variable values versus time. The time steps are shown on the horizontal axis and the variable values on the vertical axis. The plot includes markers for each time step, axis labels, a descriptive title indicating the variable and spatial

point, and a grid for readability. The y-axis formatting is controlled by the decimals parameter to ensure consistent numeric precision.

Note: Example of using *plot_variable_time_series ()*:

```
plot_variable_time_series(data_by_time,"p", point=(500,20,500))
```

Here, *data_by_time* is the data class in which the simulation results are stored (for example, *data_by_time = load_vtk_results(output_dir)*). This command plots the time series of the variable "p" from the simulation results stored in *data_by_time* at location of (500,20,500).